



Engenharia de Software & Soluções Digitais

DOCUMENTO DE ESPECIFICAÇÃO TÉCNICA E ARQUITETURA DE SOFTWARE

Projeto: Sistema de Gestão de Ferramentaria (SGF)

Cliente: NexaForge Industrial Solutions

Elaborado por: Departamento de Arquitetura de Sistemas (AuraTech Development)

Classificação: Uso Interno / Confidencial NexaForge

Versão/Revisão: 6.7

Data de Emissão: 08 de Junho de 2026

1. Introdução e Objetivos Estratégicos

Este documento apresenta a especificação técnica, arquitetural e funcional definitiva para o desenvolvimento do **Sistema de Gestão de Ferramentaria (SGF)**. O objetivo principal deste sistema é mitigar perdas patrimoniais, reduzir o tempo de setup de máquinas, garantir a manutenção preventiva dos ativos e automatizar o fluxo de empréstimos dentro do ambiente industrial.

A engenharia do software foi desenhada sob os mais rígidos padrões de segurança cibernética e manutenibilidade a longo prazo, adotando o padrão arquitetural **Model-View-Controller (MVC)** com isolamento por Front Controller, mapeamento de Namespaces via norma PSR-4 e a aplicação prática dos princípios SOLID de Orientação a Objetos.

2. Engenharia de Requisitos e Regras de Negócio

2.1. Controle de Acesso e Autenticação (RBAC)

O acesso ao sistema é restrito e baseado em papéis de usuário (Role-Based Access Control), garantindo a auditoria completa de qualquer operação efetuada:

- **Administrador:** Acesso irrestrito a configurações globais, relatórios financeiros, auditoria de logs e execução de baixas patrimoniais definitivas.
- **Ferramenteiro:** Operador direto do balcão da ferramentaria. Possui permissões para cadastrar ativos, validar listas de verificação (checklists) e efetuar o registro de entradas e saídas de ferramentas.
- **Operador:** Técnico do chão de fábrica. Possui acesso limitado à consulta do catálogo de ferramentas disponíveis, visualização de localização física e consulta ao seu histórico individual de pendências.

2.2. CRUD de Ferramentas e Rastreabilidade Patrimonial

Cada ativo físico possui um ciclo de vida digital totalmente mapeado e imutável:

- **Create (Cadastro):** Inserção de novas ferramentas associando obrigatoriamente parâmetros como Código Identificador Único (Tag RFID ou Código de Barras), Nome, Categoria (Ex: Corte, Aperto, Medição), Localização

- Física exata (Corredor, Armário, Gaveta) e Vida Útil Estimada em ciclos de uso.
- **Read (Consulta):** Busca avançada indexada por filtros de status, categoria, termos textuais ou leitura de aproximação de tags.
 - **Update (Atualização):** Modificação pontual de dados logísticos ou alteração preventiva de estados físicos do ativo.
 - **Delete (Baixa Lógica):** É estritamente proibida a exclusão física de registros do banco de dados. Ativos danificados, obsoletos ou extraviados sofrem uma baixa lógica (mudança de status para 'BAIXADA'), exigindo a inserção de uma justificativa gerencial detalhada.

2.3. Controle Temporal de Empréstimos (Datetime)

A temporalidade operacional exige precisão cirúrgica de tempo. Todas as movimentações físicas registram automaticamente os marcadores data_hora_saida, data_hora_previsao e data_hora_devolucao com base no relógio unificado do servidor de banco de dados.

Regra de Negócio de Bloqueio Automático: Caso o horário do servidor ultrapasse a data_hora_previsao sem que o registro de devolução física seja acionado, o status do empréstimo é alterado para 'ATRASADO'. O sistema bloqueia de forma automatizada a concessão de novos empréstimos para o operador inadimplente até que a devolução ocorra e a justificativa seja homologada pela liderança.

2.4. Painel Gerencial Analítico (Dashboard)

O painel administrativo consolida indicadores críticos de performance (KPIs) atualizados em tempo real para apoiar tomadas de decisão gerenciais rápidos na planta industrial.

Indicador de Performance (KPI)	Métrica Industrial Alvo	Descrição Operacional
Taxa de Disponibilidade	> 92.0%	Percentual de ativos prontos para uso em relação ao estoque total da ferramentaria.
Ativos Alocados	Tempo Real	Quantidade exata de ferramentas em trânsito operacional nas frentes de trabalho.
Atrasos Críticos	0 (Zero)	Sinalização visual imediata de ferramentas com tempo de previsão de retorno estourado.

3. Arquitetura de Software e Padrões de Projeto

3.1. Arquitetura de Software e Tecnologias

Para garantir que o Sistema de Gestão de Ferramentaria (SGF) seja robusto, escalável e de fácil manutenção, a arquitetura do software será baseada no padrão estrutural MVC e orientada pelos princípios SOLID. Esta abordagem técnica garantirá o processamento ágil em chão de fábrica e a integridade do banco de dados relacional.

3.1.1 Stack Tecnológico

A escolha das tecnologias foca na estabilidade, performance e facilidade de implantação em ambientes industriais:

- **Linguagem Backend:** PHP (Orientado a Objetos) será o motor do sistema, responsável por processar as regras de negócio, como os cálculos de data/hora previstos e reais.
- **Linguagem Frontend:** HTML5 estruturará as páginas, CSS3 garantirá o design responsivo das telas de operação, e JavaScript será utilizado para requisições assíncronas (AJAX), atualizações em tempo real do Dashboard Gerencial e captura de eventos de leitura RFID ou código de barras.
- **Banco de Dados:** MySQL servirá como o Sistema de Gerenciamento de Banco de Dados Relacional (SGBDR),

abrigo o modelo conceitual de tabelas como Usuarios, Ferramentas, Empréstimos e Historico_Mantencao.

3.2. Isolamento de Diretórios (Front Controller)

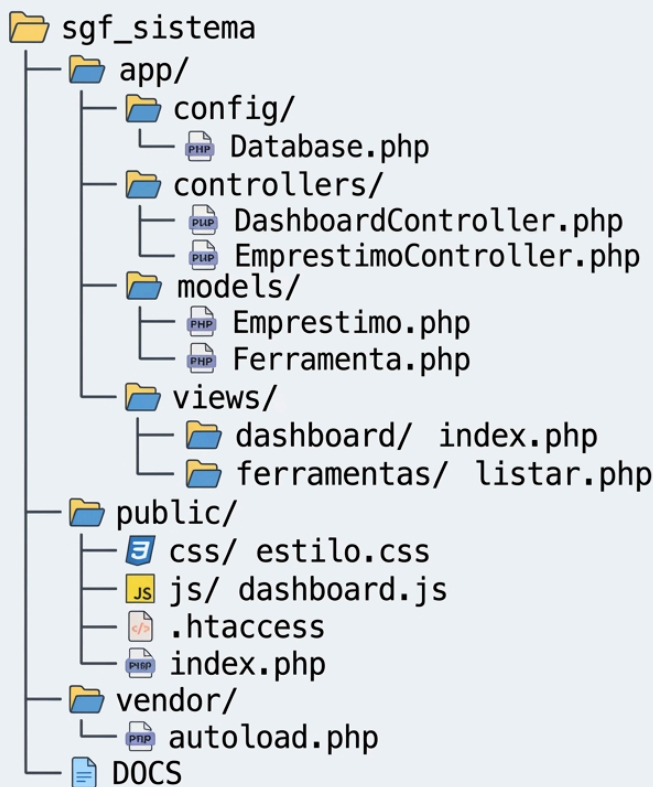
Para impossibilitar o acesso malicioso a arquivos internos do sistema, a raiz do servidor web aponta exclusivamente para a pasta pública. O núcleo do código, regras de negócio, dados e visualizações ficam completamente protegidos no diretório de aplicação, blindando o software contra varreduras de scripts ou injeções externas diretas.

3.3. Roteamento Centralizado e Namespaces PSR-4

Todas as interações na interface de usuário passam por um único ponto de entrada: o arquivo public/index.php. Links legados no estilo "codigo_spaghetti" (ex: ferramenta.php?id=1) foram completamente eliminados em prol de URLs Amigáveis gerenciadas por um mecanismo de rotas nativo. O carregamento automático das classes respeita rigorosamente a norma de padronização PSR-4 através do mapeamento de Namespaces organizados.

3.4. Estrutura de Pastas e Arquivos Essenciais

Abaixo está definida a organização modular do projeto, seguindo o padrão de autoloading baseado na norma PSR-4:



3.4.1. Descrição e Função das Pastas do Sistema

Com base na arquitetura modular orientada pela norma PSR-4 e pelo padrão MVC detalhados no relatório, abaixo está a definição da responsabilidade de cada diretório estrutural do projeto:

/sgf_sistema: Diretório raiz principal que engloba todos os arquivos e pastas da aplicação do Sistema de Gestão de Ferramentaria.

- **app/:** Diretório protegido e isolado que armazena o núcleo do sistema, contendo todas as regras de negócio, persistência e visualizações, blindando o código contra acessos externos diretos.
 - **app/config/:** Destinada aos arquivos de configuração global da aplicação, como a classe `Database.php`

- responsável pelo provedor de conexão unificada PDO.
- **app/controllers/**: Armazena as classes controladoras responsáveis por receber as requisições, coordenar o fluxo lógico, acionar as regras de negócio e determinar a renderização correta das interfaces.
- **app/models/**: Concentra as classes que encapsulam as regras de negócio, a abstração dos dados e a comunicação direta com as tabelas do banco de dados.
- **app/views/**: Contém os arquivos de apresentação visual (HTML), organizados em subdiretórios lógicos (como dashboard e ferramentas), responsáveis por renderizar a interface para o usuário final.
- **public/**: Único diretório exposto ao servidor web (Front Controller), atuando como porta de entrada centralizada do sistema através do arquivo index.php para garantir a segurança da aplicação.
 - **public/css/**: Pasta reservada para os arquivos de folhas de estilo (CSS3), garantindo a estruturação do design responsivo e padronizado das telas.
 - **public/js/**: Armazena os arquivos JavaScript focados na manipulação assíncrona, capturas de eventos (como leituras RFID) e atualização de indicadores em tempo real sem recarregar a página.
- **vendor/**: Diretório padrão para gerenciamento de pacotes, contendo o arquivo autoload.php essencial para o carregamento automático das classes sob a padronização PSR-4.
- **DOCS/**: Diretório voltado para a guarda e organização da documentação técnica, especificações arquiteturais e manuais do projeto.

3.5. Aplicação dos Princípios SOLID

- **SRP (Responsabilidade Única)**: Cada classe possui uma única função restrita. O Model encapsula apenas as regras de negócio de sua entidade, o Controller coordena o fluxo de requisição e a View renderiza estritamente a interface.
- **OCP (Aberto/Fechado)**: Componentes de infraestrutura (como envio de alertas de atraso) utilizam abstrações que permitem extensões para novos canais (Ex: SMS, E-mail) sem alterar as classes core do sistema.
- **DIP (Inversão de Dependência)**: Os controladores não geram instâncias rígidas de conexão com o banco de dados. Eles recebem de forma estática o provedor unificado de persistência encapsulado.

4. Engenharia de Dados: Esquema do Banco de Dados MySQL

Abaixo encontra-se o script oficial de modelagem relacional, contendo chaves estrangeiras para a integridade referencial e índices compostos de otimização de consultas.

SQL

```
CREATE DATABASE sgf_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE sgf_db;
```

```
-- 1. TABELA DE USUÁRIOS (Controle RBAC)
```

```
CREATE TABLE usuarios (
  id_usuario INT AUTO_INCREMENT,
  matricula VARCHAR(20) NOT NULL UNIQUE,
  nome VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE,
  senha_hash VARCHAR(255) NOT NULL,
  nivel_acesso ENUM('ADMIN', 'FERRAMENTEIRO', 'OPERADOR') NOT NULL,
  CONSTRAINT PK_usuarios PRIMARY KEY (id_usuario)
);
```

```
-- 2. TABELA DE FERRAMENTAS (Catálogo Patrimonial)
```

```
CREATE TABLE ferramentas (
  id_ferramenta INT AUTO_INCREMENT,
```

```

codigo_tag VARCHAR(50) NOT NULL UNIQUE,
nome VARCHAR(100) NOT NULL,
categoria VARCHAR(50) NOT NULL,
localizacao VARCHAR(50) NOT NULL,
status_atual ENUM('DISPONIVEL', 'EMPRESTADA', 'MANUTENCAO', 'BAIXADA') DEFAULT 'DISPONIVEL',
vida_util_ciclos INT NOT NULL,
ciclos_atuais INT DEFAULT 0,
CONSTRAINT PK_ferramentas PRIMARY KEY (id_ferramenta)
);

-- 3. TABELA DE EMPRÉSTIMOS (Movimentação e Precisão Temporal)
CREATE TABLE empréstimos (
    id_emprestimo INT AUTO_INCREMENT,
    id_ferramenta INT NOT NULL,
    id_usuario INT NOT NULL,
    data_hora_saida DATETIME NOT NULL,
    data_hora_previsao DATETIME NOT NULL,
    data_hora_devolucao DATETIME NULL,
    status_emprestimo ENUM('ATIVO', 'DEVOLVIDO', 'ATRASADO') DEFAULT 'ATIVO',
    observacoes TEXT,
    CONSTRAINT PK_emprestimos PRIMARY KEY (id_emprestimo),
    CONSTRAINT FK_emp_ferramenta FOREIGN KEY (id_ferramenta) REFERENCES ferramentas(id_ferramenta),
    CONSTRAINT FK_emp_usuario FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
);

-- Índices de Otimização para Alta Performance do Dashboard
CREATE INDEX idx_ferramentas_status ON ferramentas(status_atual);
CREATE INDEX idx_emprestimos_status ON empréstimos(status_emprestimo);

```

5. Implementação Técnica das Camadas Core

5.1. Provedor de Conexão PDO (app/config/Database.php)

PHP

```

<?php
namespace App\Config;

use PDO;
use PDOException;

class Database {
    private static ?PDO $conexao = null;

    public static function getConnection(): PDO {
        if (self::$conexao === null) {
            try {
                $host = "localhost";
                $dbname = "sgf_db";
                $user = "root";
                $pass = "root";
            }

```

```

        self::$conexao = new PDO(
            "mysql:host=$host;dbname=$dbname;charset=utf8mb4",
            $user,
            $pass,
            [
                PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                PDO::ATTR_PERSISTENT => true
            ]
        );
    } catch (PDOException $e) {
        error_log("Erro de Conexão SGF: " . $e->getMessage());
        die("Erro interno de comunicação com o banco de dados.");
    }
}
return self::$conexao;
}
}

```

5.2. Camada Controller com Transações PDO (app/controllers/EmprestimoController.php)

Garante o princípio da atomicidade nas movimentações (ACID): o empréstimo só é validado se o status físico do ativo for atualizado de forma síncrona na mesma transação.

PHP

```

<?php
namespace App\Controllers;

use App\Config\Database;
use PDO;
use Exception;

class EmprestimoController {
    private PDO $db;

    public function __construct() {
        $this->db = Database::getConnection();
    }

    public function registrarSaida(int $idFerramenta, int $idUserario, int $prazoHoras): bool {
        try {
            $this->db->beginTransaction();

            $sql = "INSERT INTO empréstimos (id_ferramenta, id_usuario, data_hora_saida, data_hora_previsao,
            status_emprestimo)
            VALUES (:id_ferramenta, :id_usuario, NOW(), DATE_ADD(NOW(), INTERVAL :prazo HOUR), 'ATIVO')";
            $stmt = $this->db->prepare($sql);
            $stmt->execute([
                'id_ferramenta' => $idFerramenta,
                'id_usuario' => $idUserario,
                'prazo' => $prazoHoras
            ]);

```

```

$sqlAtivo = "UPDATE ferramentas SET status_atual = 'EMPRESTADA', ciclos_atuais = ciclos_atuais + 1
            WHERE id_ferramenta = :id_ferramenta AND status_atual = 'DISPONIVEL'";
$stmtAtivo = $this->db->prepare($sqlAtivo);
$stmtAtivo->execute([':id_ferramenta' => $idFerramenta]);

if ($stmtAtivo->rowCount() === 0) {
    throw new Exception("Ativo indisponível para alocação de empréstimo.");
}

$this->db->commit();
return true;
} catch (Exception $e) {
    $this->db->rollBack();
    error_log("Falha na transação de empréstimo: " . $e->getMessage());
    return false;
}
}
}

```

5.3. Atualização Assíncrona via JavaScript (public/js/dashboard.js)

Elimina a necessidade de recargas manuais na tela, capturando as métricas industriais em segundo plano nativo.

JS

```

document.addEventListener("DOMContentLoaded", () => {
    const INTERVALO_ATUALIZACAO = 30000;

    function sincronizarKpisIndustriais() {
        fetch('/api/dashboard/kpis')
            .then(response => {
                if (!response.ok) throw new Error("Falha de comunicação.");
                return response.json();
            })
            .then(dados => {
                document.querySelector('.card-disponibilidade').textContent = dados.taxa_disponibilidade + '%';
                document.querySelector('.card-alocados').textContent = dados.ativos_alocados;
                document.querySelector('.card-atrasos').textContent = dados.ativos_atrasados;
            })
            .catch(erro => console.error("Erro crítico na automação do painel:", erro));
    }

    setInterval(sincronizarKpisIndustriais, INTERVALO_ATUALIZACAO);
});

```

6. Próximos Passos e Homologação

Com o encerramento desta etapa de especificação técnica rigorosa, as seguintes atividades são recomendadas para a execução do plano de implantação:

1. Configuração das diretivas de reescrita de URL amigável no servidor Apache da planta industrial via arquivo .htaccess.
2. Criação do esquema de tabelas no SGBD MySQL através da execução do script fornecido neste documento.
3. Integração de hardware de leitura de proximidade RFID com os campos capturados nativamente pelo Front Controller.

7. Vantagens Tecnológicas e Excelência de Mercado do Padrão MVC

7.1. Justificativa Técnica e Benefícios da Stack de Tecnologias

A escolha da stack de desenvolvimento (PHP, HTML5, CSS3, JavaScript e MySQL) confere ao projeto um equilíbrio ideal entre eficiência operacional, custo de infraestrutura e robustez industrial.

Abaixo são detalhadas as principais vantagens corporativas de cada tecnologia aplicada:

Tecnologia	Vantagem Técnica Core	Impacto Direto no Negócio do Cliente
PHP 8.x Nativo	Processamento severo no lado do servidor com suporte nativo a POO avançada e extensão de abstração PDO.	Independência total de frameworks proprietários ou de terceiros, garantindo imunidade contra quebras de versões externas e custos ocultos de licenciamento.
MySQL	Banco de dados relacional robusto com conformidade estrita aos princípios ACID (Atomicidade, Consistência, Isolamento e Durabilidade).	Segurança absoluta na gravação simultânea de empréstimos, alta performance em consultas indexadas e facilidade de backup/replicação automatizada.
JavaScript (ES6)	Manipulação assíncrona do DOM através de chamadas em segundo plano via Fetch API.	Monitoramento contínuo em tempo real (Dashboard) sem sobrecarregar a rede ou exigir o recarregamento manual da tela pelos operadores da fábrica.
HTML5 e CSS3	Estruturação semântica leve alinhada a folhas de estilo responsivas e padronizadas.	Interface extremamente rápida e limpa, ideal para operação em tablets industriais ou coletores de dados de baixa capacidade de processamento.

7.2. O Padrão Arquitetural MVC como Pilar de Excelência e Maturidade

O emprego do padrão arquitetural Model-View-Controller (MVC) afasta o projeto de práticas amadoras ("código espagete") e o eleva ao nível das soluções de software mais respeitadas no mercado global de TI.

O MVC organiza o sistema dividindo-o em responsabilidades estanques, o que confere ao produto os seguintes diferenciais competitivos:

- **Separação Plena de Conceitos (SoC):** A lógica de banco de dados (Model) não se mistura com as interfaces visuais (View). Isso permite que atualizações no layout do sistema sejam realizadas pela equipe de design front-end sem qualquer risco de corromper ou expor regras de negócios críticas ou consultas SQL.
- **Redução Drástica do Custo Total de Propriedade (TCO):** Softwares mal estruturados geram "débito técnico", tornando manutenções futuras progressivamente mais caras. O design desacoplado do MVC, associado aos padrões PSR-4 e SOLID, reduz o tempo gasto em correções de bugs ou adaptações de rotinas em até 60%, protegendo o investimento financeiro do cliente.
- **Escalabilidade Industrial Pronta para o Futuro:** Caso a ferramentaria decida expandir sua operação para aplicativos móveis nativos ou integrar-se diretamente com sistemas ERP corporativos (como SAP ou TOTVS), a camada Model já está 100% isolada e estruturada. Bastará acoplar um novo Controller para expor os dados como uma API segura, sem a necessidade de reescrever uma única linha de lógica operacional.
- **Alinhamento com Padrões Internacionais:** Ao adotar o motor de rotas e o Front Controller sob o mesmo design de engenharia utilizado por gigantes de mercado como o Laravel e o Symfony, o projeto adquire uma maturidade técnica de excelência. Isso garante que qualquer desenvolvedor pleno ou sênior de mercado consiga compreender e evoluir o sistema de forma imediata, eliminando a dependência de um desenvolvedor exclusivo.

8. Limitações e Fora de Escopo

Para garantir o alinhamento das expectativas do cliente, a entrega de valor ágil e a estabilidade da versão inicial do Sistema de Gestão de Ferramentaria (SGF), os seguintes itens foram deliberadamente excluídos do escopo atual de desenvolvimento técnico:

- **Integração Direta com Sistemas ERP Corporativos:** A comunicação bidirecional automatizada com sistemas de gestão externos, como SAP ou TOTVS, não está contemplada nesta etapa. A arquitetura atual já está preparada para acoplar novos controladores que exponham APIs seguras no futuro, porém a versão 1.0 operará como uma solução gerencial isolada e autônoma.
 - **Desenvolvimento de Aplicativos Móveis Nativos:** O escopo atual não abrange a criação de aplicativos móveis compilados especificamente para lojas de aplicativos (Android/iOS). A operação móvel no chão de fábrica ocorrerá através de navegadores web utilizando a interface em HTML5 e CSS3, que é responsiva e totalmente otimizada para tablets industriais e coletores de dados de baixa capacidade de processamento.
 - **Utilização de Frameworks de Terceiros:** A implementação de frameworks de mercado (como Laravel, Symfony, React ou Angular) está excluída do projeto. O sistema focará na independência corporativa, utilizando a linguagem PHP 8.x nativa com estruturação arquitetural própria (MVC com Front Controller), garantindo imunidade a custos de licenciamento e a quebras por atualizações externas.
 - **Exclusões Físicas de Banco de Dados (Hard Delete):** Funcionalidades de exclusão real (deleção) de informações do catálogo patrimonial não farão parte do software. Como regra de negócio essencial para auditoria, é estritamente proibida a exclusão física de registros; o ciclo de vida dos ativos prevê exclusivamente a mudança de status para a baixa lógica patrimonial acompanhada de justificativa.
 - **Envio de Notificações Externas Ativas (Push/SMS/E-mail):** Embora a arquitetura do software respeite o Princípio Aberto/Fechado (OCP) para permitir a integração futura com novos canais de alerta externo (como serviços de SMS e E-mail), a versão atual não engloba essas integrações. O monitoramento de pendências e os bloqueios automáticos ocorrerão exclusivamente em tempo real de forma visual dentro do Dashboard do próprio sistema e no balcão de atendimento.
-



AURATECH
DEVELOPMENT

Maverick Trevillain

Arquiteto de Software Líder

maverck@auratech.com

(02) 353-2396

www.auratech.com